

B 2.1 Programming fundamentals Python



B2.1 Programming fundamentals

This section explores the foundational elements of programming, including variables, data types, string manipulation, exception handling, and debugging techniques. Understanding these concepts is crucial for writing effective and robust programs

Last updated: 17 Sep 2025

B2.1 Programming Fundamentals

This section explores the foundational elements of programming, including variables, data types, string manipulation, exception handling, and debugging techniques. Understanding these concepts is crucial for writing effective and robust programs

B2.1.1 Variables and Data Types

Variables are used to store data in a program. Each variable has a data type that determines the kind of data it can hold. Common data types include:

Integer: Whole numbers (e.g., 10, -5, 0)

Decimal: Numbers with fractional parts (e.g., 3.14, -2.5)

Character: Single letters or symbols (e.g., 'A', '\$', '?')

String: Sequences of characters (e.g., "Hello", "World")

Boolean: Represents true or false values

Code Examples:

Python:

```
age = 25
price = 19.99
initial = 'J'
name = "John Doe"
isAdult = True
```

Global vs. Local Variables:

Global variables: Declared outside of any function and can be accessed from anywhere in the program.

Local variables: Declared inside a function and can only be accessed within that function.

B2.1.2 String Manipulation

Strings are often manipulated in programs to extract information or modify text. Common string manipulation techniques include:

Substring extraction: Getting a portion of a string.

Concatenation: Joining two or more strings together.

Replacement: Replacing part of a string with another string.

Code Examples:

Python:

```
text = "Hello World"
sub = text[0:5] # sub = "Hello"
concat = text + "!" # concat = "Hello World!"
replaced = text.replace("World", "Python") # replaced = "Hello Python"
```

B2.1.3 Exception Handling

Exceptions are errors that occur during program execution. Exception handling helps to prevent programs from crashing when errors occur.

Common Exception Handling Techniques:

Try-catch blocks (Java) / Try-except blocks (Python): Code that might cause an exception is placed in the try block. If an exception occurs, it is caught and handled in the catch or except block.

Finally block: Code in the finally block is always executed, regardless of whether an exception occurs or not.

Code Examples:

Python:

```
try:
    result = 10 / 0 # This will cause a ZeroDivisionError
except ZeroDivisionError as e:
    print("Error: Division by zero.")
finally:
    print("This will always execute.")
```

B2.1.4 Debugging Techniques

Debugging is the process of finding and fixing errors in a program. Common debugging techniques include:

Trace tables: Manually tracking the values of variables as the program executes.

Breakpoints: Pausing the program execution at specific points to examine the program's state.

Print statements: Adding print statements to output the values of variables or the flow of execution.

Step-by-step code execution: Executing the code one line at a time to observe the program's behaviour.

Example: Using print statements to debug a Python program:

```
def calculate_sum(a, b):  
    print(f"a = {a}, b = {b}") # Print the values of a and b  
    result = a + b  
    print(f"result = {result}") # Print the calculated result  
    return result  
  
calculate_sum(5, 3)
```

This will output:

```
a = 5, b = 3  
result = 8
```

By examining the output, you can track the values of variables and identify potential errors.

Conclusion:

This section has covered the essential programming fundamentals, including variables, data types, string manipulation, exception handling, and debugging techniques. Mastering these concepts will provide a solid foundation for writing more complex and robust programs.

1

What is the output of the following code?

```
text = "Python"
```

```
print(text[1:4])
```

- A. ☐ Pyt
- B. ☐ ytho
- C. ☒ yth
- D. ☐ thon

2

Which data type is used to store whole numbers?

A. ☐ bool

B. ☐ float

C. ☐ str

D. ☒ int

3

What is the purpose of the try-except block in Python?

A. ☐ To define a function.

B. ☒ To handle exceptions.

C. ☐ To create a loop.

D. ☐ To declare variables.

4

What is the output of the following code?

try:

```
result = 10 / 0
```

except ZeroDivisionError:

```
print("Error!")
```

A. ☐ 10

B. ☐ 0

C. ☒ Error!

D. ☐ Infinity

5

Which of the following is NOT a debugging technique?

A. ☒ Compiling the code

B. ☐ Using a trace table

C. ☐ Using breakpoints

D. ☐ Adding print statements

6

What is the correct way to concatenate two strings in Python?

A. ☐ string1 & string2

B. ☐ string1.concat(string2)

C. ☒ string1 + string2

D. ☐ String1.append(string2)

7

What is the output of the following code?

```
name = "Alice"
```

```
age = 30
```

```
print(f"My name is {name} and I am {age} years old.")
```

A. ☒ My name is Alice and I am 30 years old.

- B. ☐ My name is name and I am age years old.
- C. ☐ Error
- D. ☐ My name is {name} and I am {age} years old.

8

What is the purpose of a finally block in exception handling?

- A. ☐ To execute code only if an exception occurs.
- B. ☒ To execute code regardless of whether an exception occurs or not.
- C. ☐ To catch all types of exceptions.
- D. ☐ To define a new exception.

9

What does the following code do?

```
text = "hello world"  
print(text.replace("world", "Python"))
```

- A. ☐ Prints "world"
- B. ☒ Prints "hello Python"
- C. ☐ Prints "Python"
- D. ☐ Prints "hello world"

10

Which of the following is a valid variable name in Python?

- A. ☐ 1st_name

B. ☐ first_name

C. ☐ first-name

D. ☐ first name

All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.

 **Report a problem**